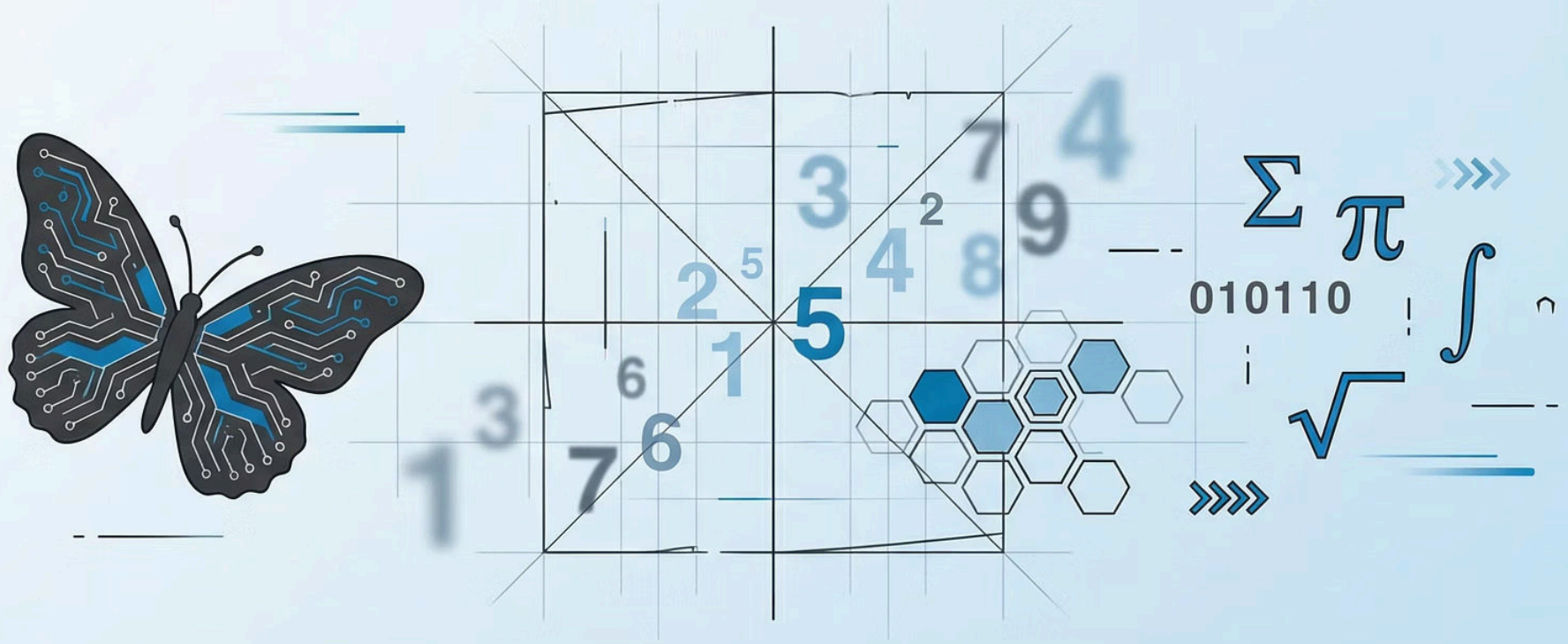
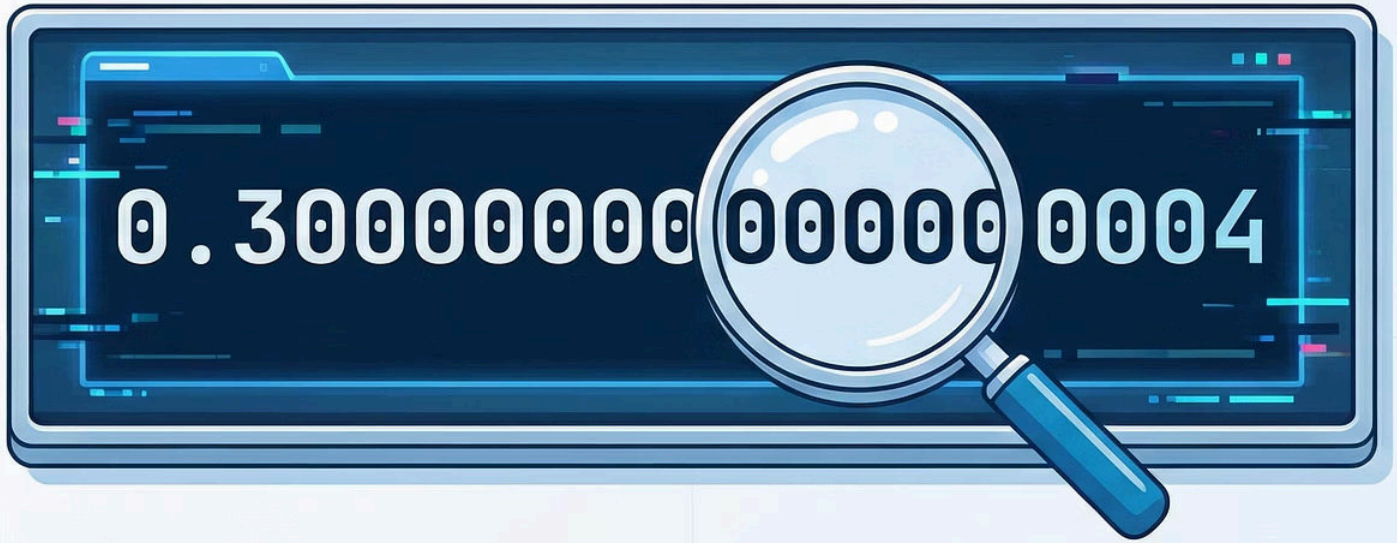
误差的蝴蝶效应与科学审视

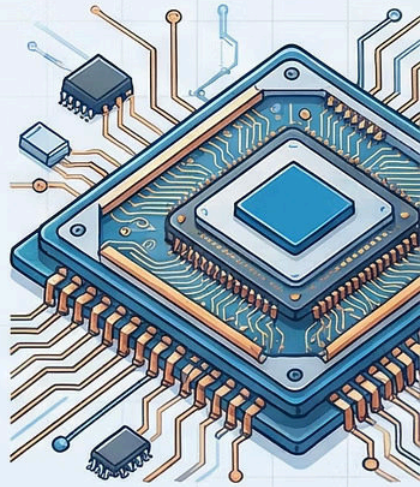
浮点精度陷阱的深度剖析与航天级防御实践



0.1 + 0.2 ≠ 0.3

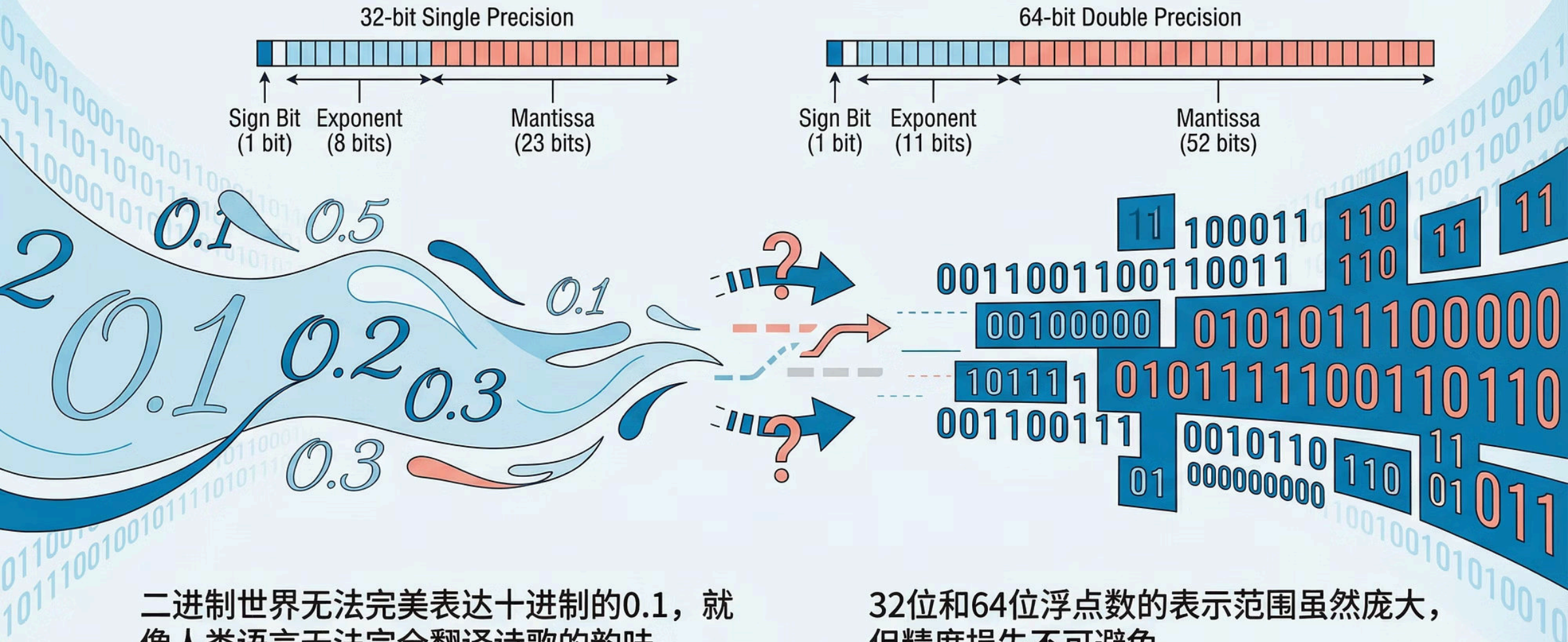






- 这一反直觉的计算结果，绝非计算机的随机故障，而是IEEE 754标准下二进制近似表示的必然产物。
- 每一个看似简单的小数计算，都隐藏着精度损失的陷阱。
- 深入理解浮点数原理，是每位程序员的必修课。

IEEE 754标准的宿命



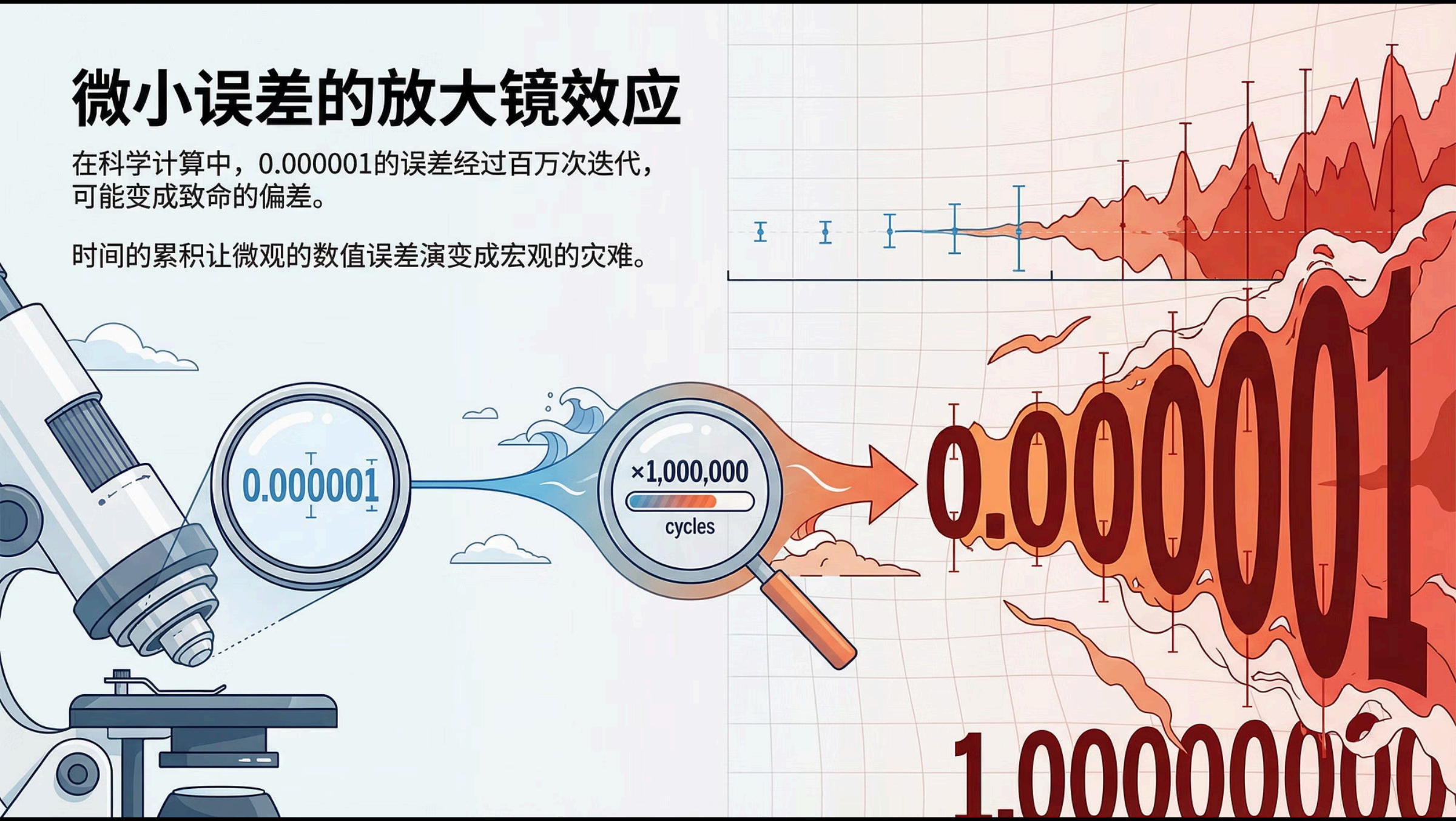
二进制世界无法完美表达十进制的0.1，就像人类语言无法完全翻译诗歌的韵味。

32位和64位浮点数的表示范围虽然庞大，但精度损失不可避免。

微小误差的放大镜效应

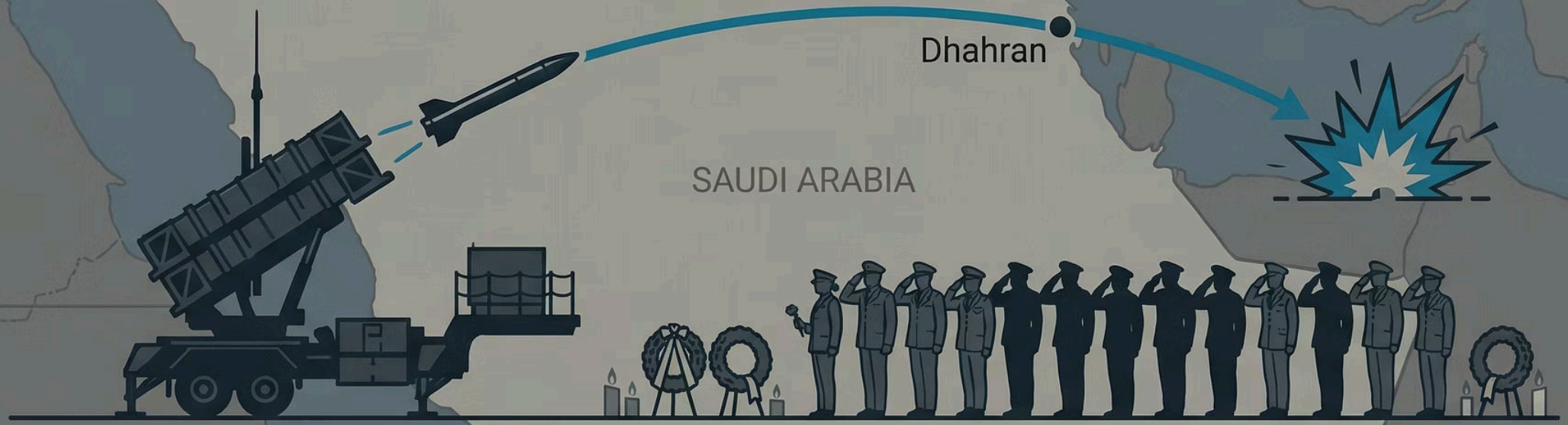
在科学计算中，0.000001的误差经过百万次迭代，可能变成致命的偏差。

时间的累积让微观的数值误差演变成宏观的灾难。



爱国者导弹的惨痛历史

1991.2.25



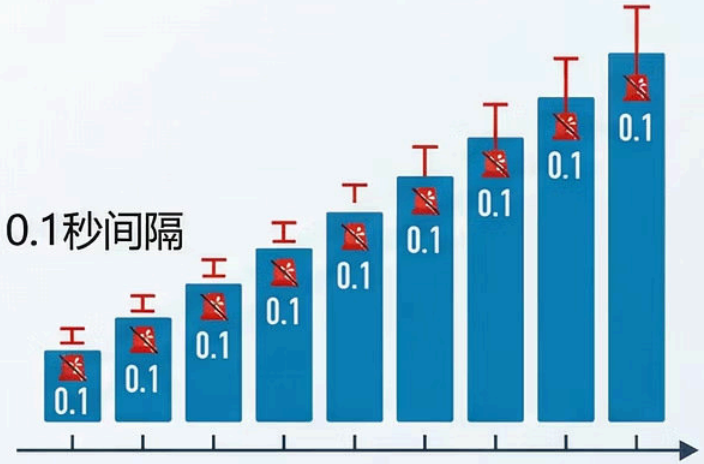
1991年2月25日，沙特阿拉伯达兰，0.34秒的累积误差导致拦截失败，28名美军士兵阵亡。

这不是技术故障，而是对浮点精度认知不足的代价。

误差累积的死亡螺旋

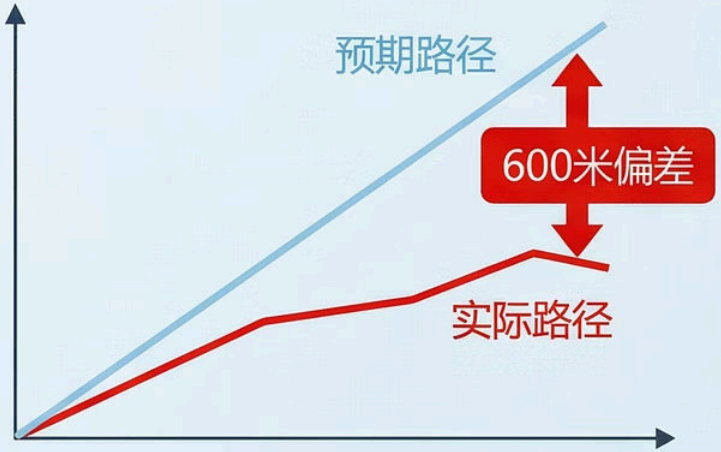
爱国者系统每0.1秒更新一次时间，但0.1无法被二进制精确表示。





0.000000095
×
360,000（100小时）
=
= 0.34 秒

经过100小时运行，累积误差达到0.34秒，
导致目标位置偏差600米。



28个生命的警示

每一行代码背后都可能关乎生命，精度不是技术细节，而是道德责任。

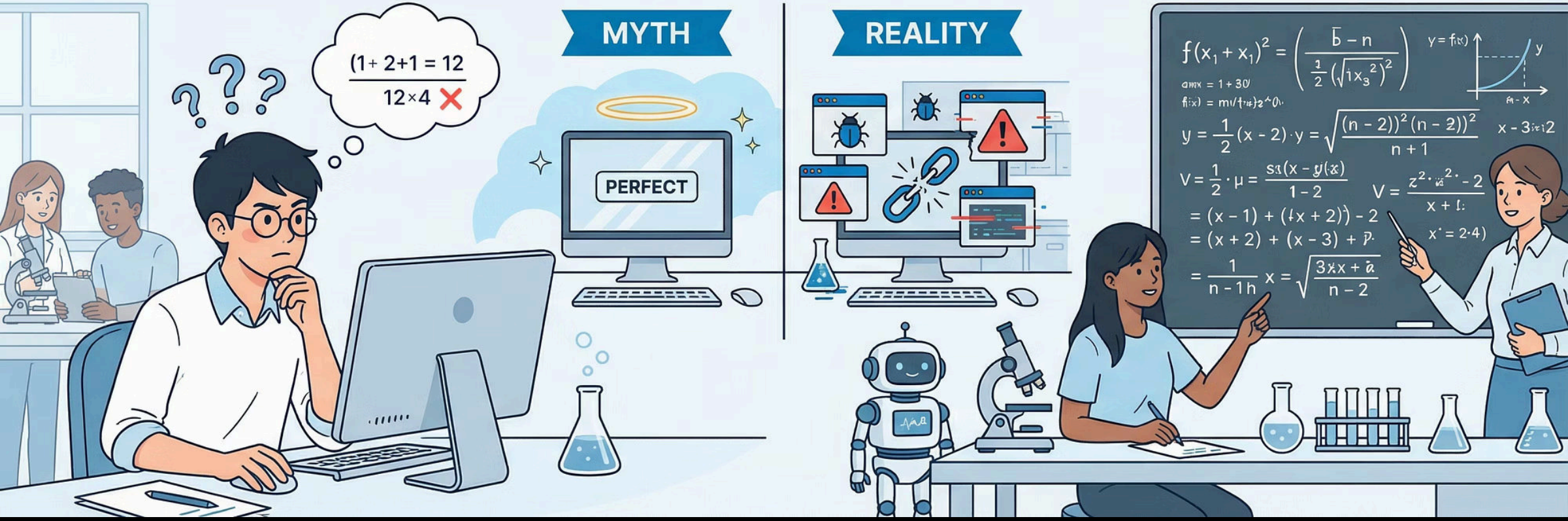
这场悲剧永远提醒我们：在关键系统中，没有细枝末节。

```
float result = 0.0;
for(int i=0; i<28; i++) {
    result += data[i] / 3.14;
}
```



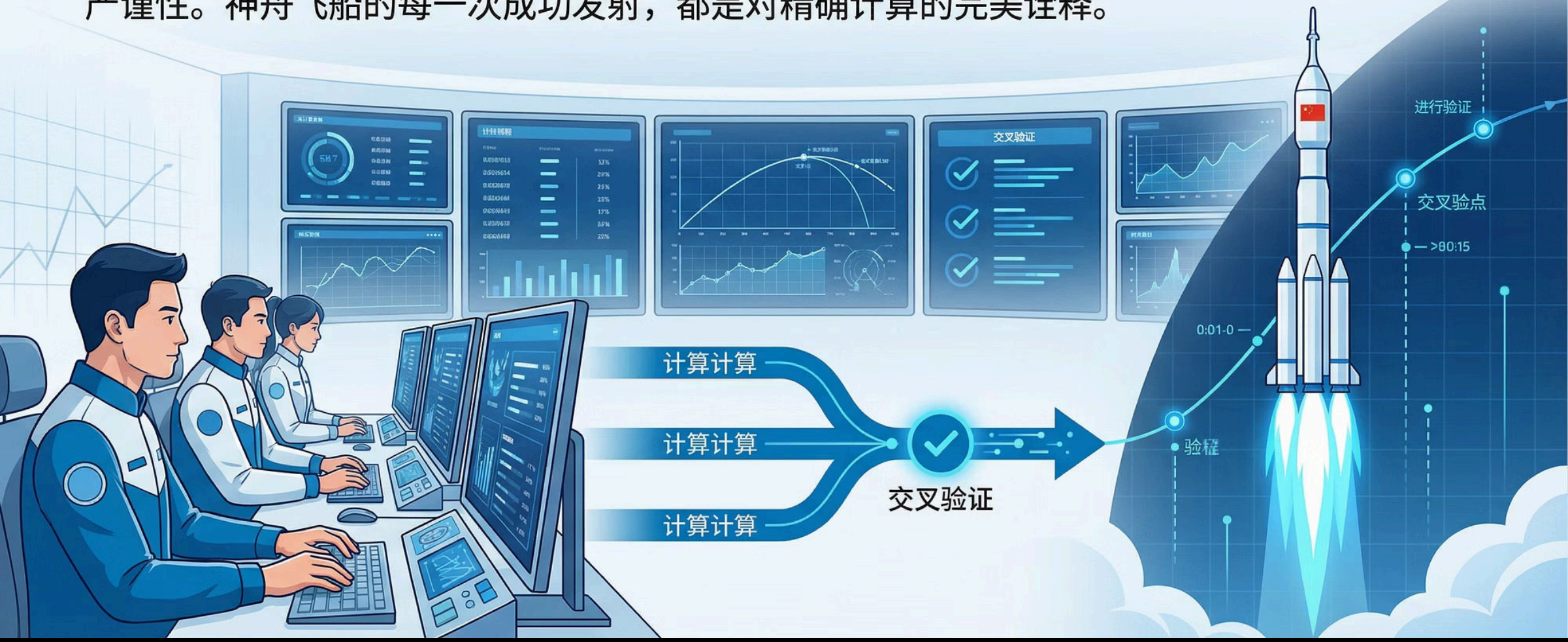
摒弃计算机迷信

我们必须引导学生摒弃对计算机结果的盲目信任，培养批判性思维和验证能力，让技术成为工具而非权威。



中国航天人的智慧

像中国航天人一样，通过三重冗余计算与交叉验证，在代码的微观世界里捍卫真理的严谨性。神舟飞船的每一次成功发射，都是对精确计算的完美诠释。



理论与实践的结合



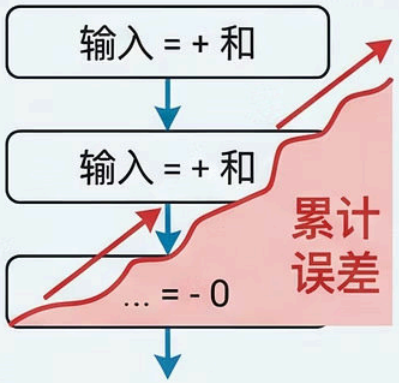
真正的工程师不仅要掌握算法原理，
更要具备发现和解决实际问题的能力。

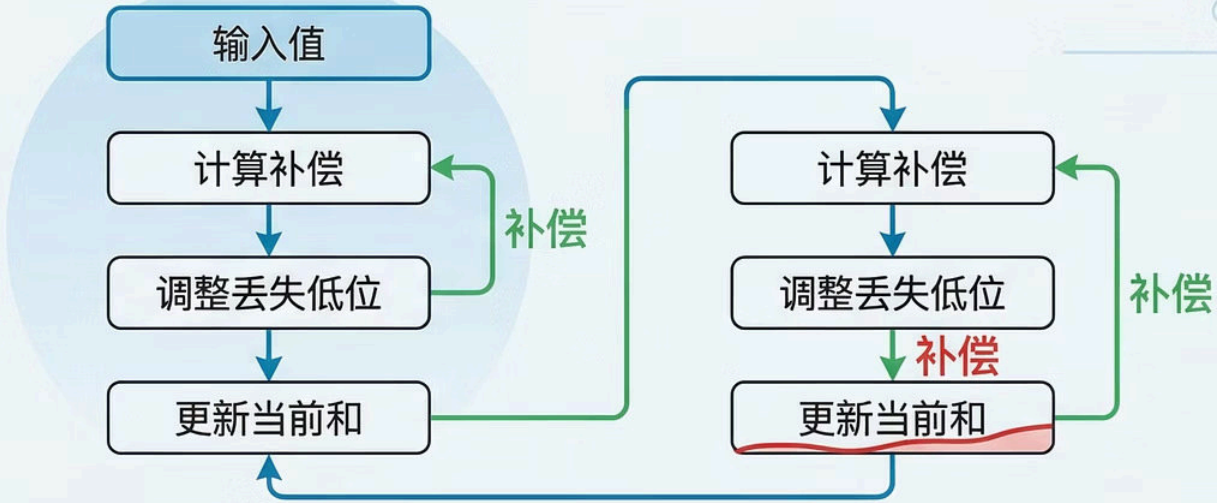
每一次数值验证都是对科学严谨性的坚持。

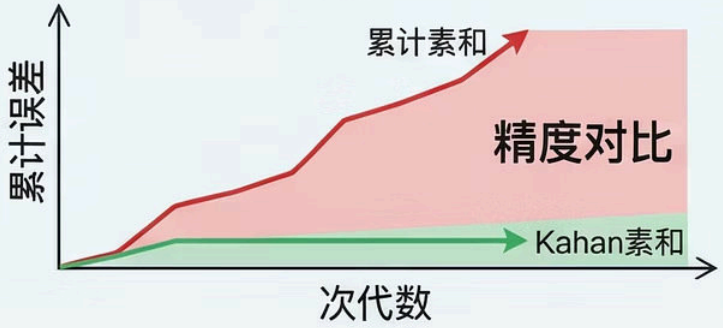
理论预测	实验结果
• x = 0.37 • y = 0.33 ✓	• x = 0.80 • y = 0.22 ✓
• x = 0.1 • x = 0.2 • z = 0.23 • z = 0.0 	 ✓

Kahan求和算法的智慧

通过剖析Kahan求和算法如何利用补偿变量消除累积误差。这是从算法层面规避精度损失的.

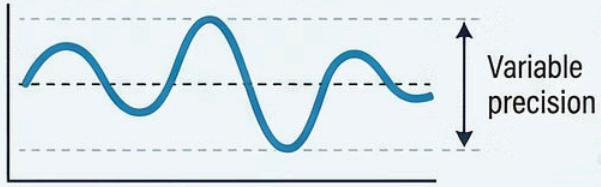






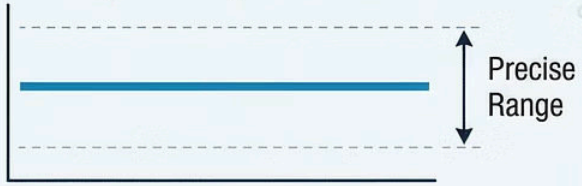
定点数的嵌入式应用

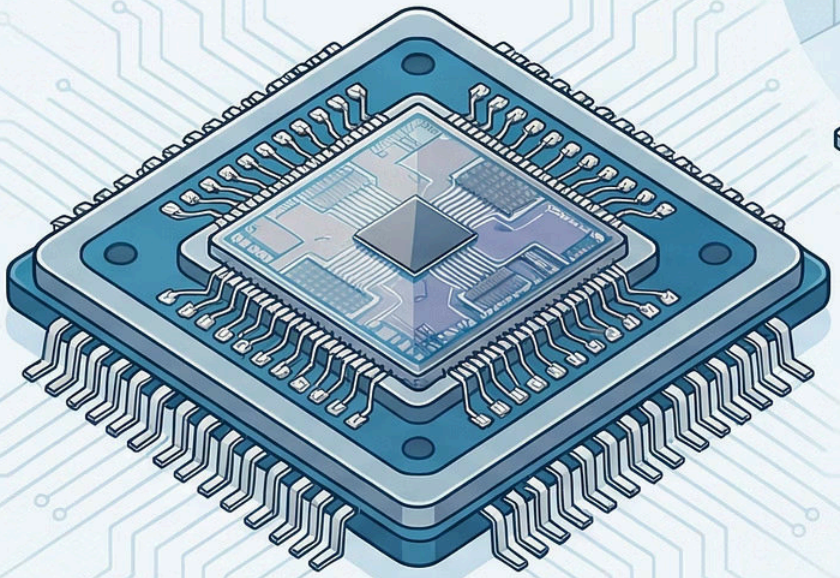
Floating-point Representation



在嵌入式系统中，定点数提供了更可控的精度保证。

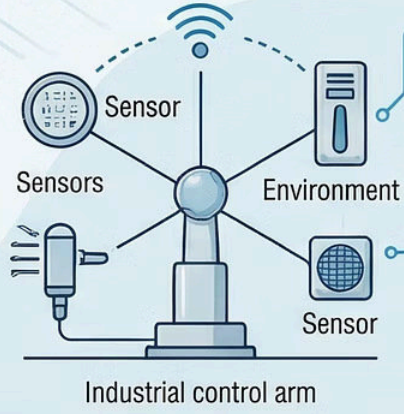
Fixed-point Representation





	Speed	Precision	Power Consumption
Floating-point	✓		0.0%
	✓		0.6%
Fixed-point	✓		**8.8%**

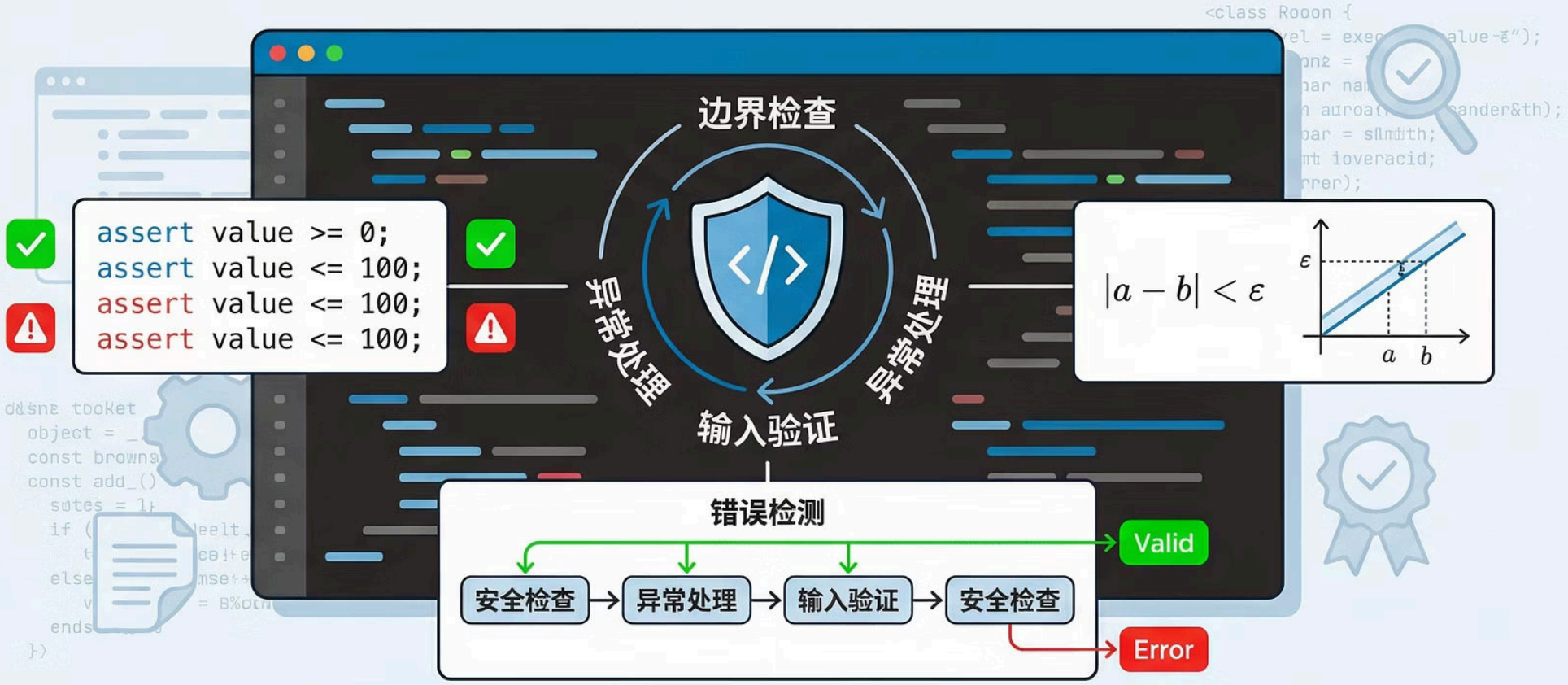
当性能和精度都至关重要时，定点数是更明智的选择。





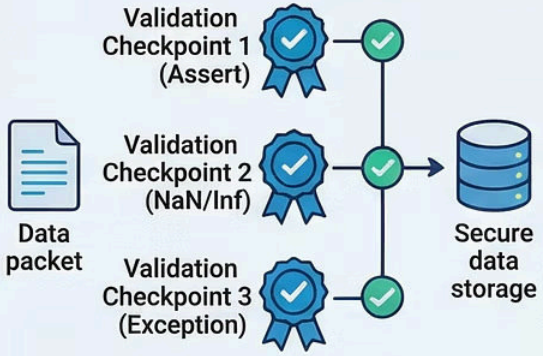
防御性编程的职业伦理

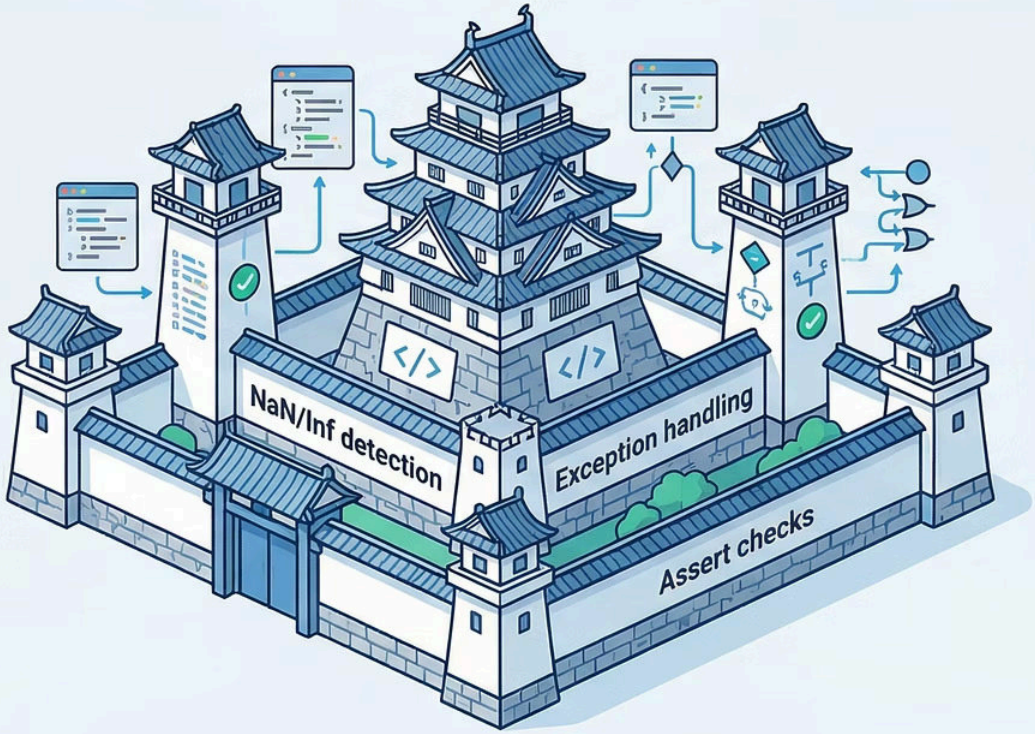
防御性编程是职业工程师的必修课：在关键计算中强制引入断言检查。使用Epsilon容差替代直接相等比较，这是对代码质量的承诺。



Assert检查的护城河

Assert检查NaN与Inf异常值，这些技术规范不仅是代码的护城河。更是每一位开发者对





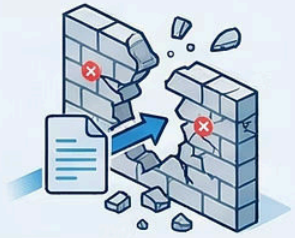

```
assert(!std::isnan(value),
   "Value must not be NaN");

assert(!std::isinf(value),
   "Value must not be infinite");

// Safe processing...
```

Vulnerable Code

```
double result = calculate(data);
// No checks
```



Defended Code

```
double result = calculate(data);
if (isValid(result)) {
  // Safe
}
```





代码质量关乎生命

在人工智能和自动驾驶的时代，精确计算的重要性前所未有。

每一位程序员都是数字世界的守护者，肩负着技术伦理的重责。